In [2]:
```
%load_ext sql
```

In [3]:
```
%sql sqlite://
```

In [4]:
```
%%sql
CREATE TABLE Worker (
        WORKER_ID INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
        FIRST_NAME CHAR(255),
        LAST_NAME CHAR(255),
        SALARY INT,
        JOINING_DATE DATETIME,
        DEPARTMENT CHAR(25)
);
```

```
 * sqlite://
Done.
```

Out[4]: []

In [5]:
```
%%sql
INSERT INTO Worker
        (WORKER_ID, FIRST_NAME, LAST_NAME, SALARY, JOINING_DATE, DEPARTMENT)
                (1, 'Monika', 'Arora', 100000, '21-02-20 09.00.00', 'HR'),
                (2, 'Niharika', 'Verma', 80000, '21-06-11 09.00.00', 'Admin'
                (3, 'Vishal', 'Singhal', 300000, '21-02-20 09.00.00', 'HR'),
                (4, 'Amitabh', 'Singh', 500000, '21-02-20 09.00.00', 'Admin'
                (5, 'Vivek', 'Bhati', 500000, '21-06-11 09.00.00', 'Admin'),
                (6, 'Vipul', 'Diwan', 200000, '21-06-11 09.00.00', 'Account'
                (7, 'Satish', 'Kumar', 75000, '21-01-20 09.00.00', 'Account'
                (8, 'Geetika', 'Chauhan', 90000, '21-04-11 09.00.00', 'Admir
        (9, 'ABHISHEK', 'RAJ',10000000, '21-04-11 09.00.00', 'Admin'),
        (10, 'ABHISHEK', 'RAJ',10000000, '21-04-11 09.00.00', 'HR');
```

```
 * sqlite://
10 rows affected.
```

Out[5]: []

In [6]:
```
%%sql
INSERT INTO Worker
        (FIRST_NAME, LAST_NAME, SALARY, JOINING_DATE, DEPARTMENT) VALUES
                ('Raj', 'Arora', 1000000, '24-02-20 09.00.00', 'HR'),
        ('Abhimanyu K', 'Verma', 800000, '24-06-11 09.00.00', 'masterAdminfu
        ('Abhimanyu', 'Verma', 800000, '24-06-11 09.00.00', 'Admin');
```

```
 * sqlite://
3 rows affected.
```

Out[6]: []

In [7]:
```
%%sql
select * from Worker;
```

```
 * sqlite://
Done.
```

Out[7]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMEN
1	Monika	Arora	100000	21-02-20 09.00.00	H
2	Niharika	Verma	80000	21-06-11 09.00.00	Admi
3	Vishal	Singhal	300000	21-02-20 09.00.00	H
4	Amitabh	Singh	500000	21-02-20 09.00.00	Admi
5	Vivek	Bhati	500000	21-06-11 09.00.00	Admi
6	Vipul	Diwan	200000	21-06-11 09.00.00	Accour
7	Satish	Kumar	75000	21-01-20 09.00.00	Accour
8	Geetika	Chauhan	90000	21-04-11 09.00.00	Admi
9	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	Admi
10	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	H
11	Raj	Arora	1000000	24-02-20 09.00.00	H
12	Abhimanyu K	Verma	800000	24-06-11 09.00.00	masterAdminfu
13	Abhimanyu	Verma	800000	24-06-11 09.00.00	Admi

In [8]:
```
%%sql
CREATE TABLE Bonus (
        WORKER_REF_ID INT,
        BONUS_AMOUNT INT(10),
        BONUS_DATE DATETIME,
        FOREIGN KEY (WORKER_REF_ID)
                REFERENCES Worker(WORKER_ID)
        ON DELETE CASCADE
);
```

```
 * sqlite://
Done.
```

Out[8]: []

In [9]:
```
%%sql
INSERT INTO Bonus
        (WORKER_REF_ID, BONUS_AMOUNT, BONUS_DATE) VALUES
                (001, 5000, '23-02-20'),
                (002, 3000, '23-06-11'),
                (003, 4000, '23-02-20'),
```

```
                (001, 4500, '23-02-20'),
                (002, 3500, '23-06-11');
```

```
 * sqlite://
5 rows affected.
```

Out[9]: []

In [10]:
```
%%sql
CREATE TABLE Title (
        WORKER_REF_ID INT,
        WORKER_TITLE CHAR(25),
        AFFECTED_FROM DATETIME,
        FOREIGN KEY (WORKER_REF_ID)
                REFERENCES Worker(WORKER_ID)
        ON DELETE CASCADE
);
```

```
 * sqlite://
Done.
```

Out[10]: []

In [11]:
```
%%sql
INSERT INTO Title
        (WORKER_REF_ID, WORKER_TITLE, AFFECTED_FROM) VALUES
 (001, 'Manager', '2023-02-20 00:00:00'),
 (002, 'Executive', '2023-06-11 00:00:00'),
 (008, 'Executive', '2023-06-11 00:00:00'),
 (005, 'Manager', '2023-06-11 00:00:00'),
 (004, 'Asst. Manager', '2023-06-11 00:00:00'),
 (007, 'Executive', '2023-06-11 00:00:00'),
 (006, 'Lead', '2023-06-11 00:00:00'),
 (003, 'Lead', '2023-06-11 00:00:00');
```

```
 * sqlite://
8 rows affected.
```

Out[11]: []

Q-1. Write an SQL query to fetch “FIRST_NAME” from the Worker table using the alias name <WORKER_NAME>.

Hint: An alias is a user-friendly label for a column in the SQL table. Use the ‘As’ clause to specify the alias in the query.

In [12]:
```
%%sql
Select FIRST_NAME AS WORKER_NAME from Worker;
```

```
 * sqlite://
Done.
```

Out[12]:

WORKER_NAME
Monika
Niharika
Vishal
Amitabh
Vivek
Vipul
Satish
Geetika
ABHISHEK
ABHISHEK
Raj
Abhimanyu K
Abhimanyu

Q-2. Write an SQL query to fetch “FIRST_NAME” from the Worker table in upper case.

Hint: SQL provides string functions to change the case. In this case, you can call the upper() function.

In [13]:
```
%%sql
Select upper(FIRST_NAME) from Worker;
```

```
 * sqlite://
Done.
```

Out[13]:

upper(FIRST_NAME)
MONIKA
NIHARIKA
VISHAL
AMITABH
VIVEK
VIPUL
SATISH
GEETIKA
ABHISHEK
ABHISHEK
RAJ
ABHIMANYU K
ABHIMANYU

Q-3. Write an SQL query to fetch unique values of DEPARTMENT from the Worker table.

Hint: Use the DISTINCT keyword to list the unique values of a SQL table column removing the duplicates.

In [14]:
```
%%sql
Select distinct DEPARTMENT from Worker;
```

```
 * sqlite://
Done.
```

Out[14]:

DEPARTMENT
HR
Admin
Account
masterAdminful

Q-4. Write an SQL query to print the first three characters of FIRST_NAME from the Worker table.

As mentioned earlier, SQL supports inline string operations like SUBSTRING(). Use this function on the column name to extract a part of it

In [15]:
```
%%sql
Select substring(FIRST_NAME,1,3) from Worker;
```

```
 * sqlite://
Done.
```

Out[15]:

substring(FIRST_NAME,1,3)
Mon
Nih
Vis
Ami
Viv
Vip
Sat
Gee
ABH
ABH
Raj
Abh
Abh

Q-5. Write an SQL query to find the position of the alphabet (‘a’) in the first_name column ‘Monika’ from the Worker table.

Hint: CHARINDEX(substring, string, [start_position])

SELECT CHARINDEX('a', FIRST_NAME) AS PositionOfA FROM Worker WHERE FIRST_NAME ='Monika';

The CHARINDEX() function searches for a substring in a string, and returns the position. If the substring is not found, this function returns 0. Note: This function performs a case-insensitive search.

substring: The substring you want to find within the string. Here "a"

string: The string in which you want to search for the substring. Here "Monika"

start_position (optional): The position in the string to start the search. If omitted, the search starts at the beginning of the string

Note:The INSTR does a case-insensitive search. Using the BINARY operator will make INSTR work as the case-sensitive function.

In [16]:
```
%%sql

Select INSTR(FIRST_NAME,'a')
from Worker
where FIRST_NAME = 'Monika';
```

```
 * sqlite://
Done.
```

Out[16]:

INSTR(FIRST_NAME,'a')
6

Q-6. Write an SQL query to print the FIRST_NAME from the Worker table after removing white spaces from the right side.

In [17]:
```
%%sql
Select RTRIM(FIRST_NAME) from Worker;
```

```
 * sqlite://
Done.
```

Out[17]:

RTRIM(FIRST_NAME)
Monika
Niharika
Vishal
Amitabh
Vivek
Vipul
Satish
Geetika
ABHISHEK
ABHISHEK
Raj
Abhimanyu K
Abhimanyu

Q-7. Write an SQL query to print the DEPARTMENT from the Worker table after removing white spaces from the left side.

In [18]:
```
%%sql
Select LTRIM(DEPARTMENT) from Worker;
```

```
 * sqlite://
Done.
```

Out[18]:

LTRIM(DEPARTMENT)
HR
Admin
HR
Admin
Admin
Account
Account
Admin
Admin
HR
HR
masterAdminful
Admin

Q-8. Write an SQL query that fetches the unique values of DEPARTMENT from the Worker table and prints its length.

In [19]:
```
%%sql
Select distinct DEPARTMENT, length(DEPARTMENT) AS STRINGLENGHT  from Worker;
```

```
 * sqlite://
Done.
```

Out[19]:

DEPARTMENT	STRINGLENGHT
HR	2
Admin	5
Account	7
masterAdminful	14

Q-9. Write an SQL query to print the FIRST_NAME from the Worker table after replacing ‘a’ with ‘A’.

In [20]:
```
%%sql
Select REPLACE(FIRST_NAME,'a','A') from Worker;
```

```
 * sqlite://
Done.
```

Out[20]:

REPLACE(FIRST_NAME,'a','A')
MonikA
NihArikA
VishAl
AmitAbh
Vivek
Vipul
SAtish
GeetikA
ABHISHEK
ABHISHEK
RAj
AbhimAnyu K
AbhimAnyu

Q-10. Write an SQL query to print the FIRST_NAME and LAST_NAME from the Worker table into a single column COMPLETE_NAME. A space char should separate them.

Hint: To join the first and last names with a space in between, you can use the || operator, which works like a glue for text in SQL. You can also use MySQL CONCAT() function.

MS SQL SERVER COMMAND

Select CONCAT(FIRST_NAME, ' ', LAST_NAME) AS 'COMPLETE_NAME' from Worker;

In [21]:
```
%%sql
SELECT FIRST_NAME || ' ' || LAST_NAME AS COMPLETE_NAME FROM Worker;
```

```
 * sqlite://
Done.
```

Out[21]:

COMPLETE_NAME
Monika Arora
Niharika Verma
Vishal Singhal
Amitabh Singh
Vivek Bhati
Vipul Diwan
Satish Kumar
Geetika Chauhan
ABHISHEK RAJ
ABHISHEK RAJ
Raj Arora
Abhimanyu K Verma
Abhimanyu Verma

Q-11. Write an SQL query to print all Worker details from the Worker table order by FIRST_NAME Ascending

In [22]:
```
%%sql
Select * from Worker order by FIRST_NAME asc;
```

```
 * sqlite://
Done.
```

Out[22]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMEN
9	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	Admi
10	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	H
13	Abhimanyu	Verma	800000	24-06-11 09.00.00	Admi
12	Abhimanyu K	Verma	800000	24-06-11 09.00.00	masterAdminfu
4	Amitabh	Singh	500000	21-02-20 09.00.00	Admi
8	Geetika	Chauhan	90000	21-04-11 09.00.00	Admi
1	Monika	Arora	100000	21-02-20 09.00.00	H
2	Niharika	Verma	80000	21-06-11 09.00.00	Admi
11	Raj	Arora	1000000	24-02-20 09.00.00	H
7	Satish	Kumar	75000	21-01-20 09.00.00	Accour
6	Vipul	Diwan	200000	21-06-11 09.00.00	Accour
3	Vishal	Singhal	300000	21-02-20 09.00.00	H
5	Vivek	Bhati	500000	21-06-11 09.00.00	Admi

Q-12. Write an SQL query to print all Worker details from the Worker table order by FIRST_NAME Ascending and DEPARTMENT Descending.

In [23]:
```
%%sql
Select * from Worker order by FIRST_NAME asc,DEPARTMENT desc;
```

```
 * sqlite://
Done.
```

Out[23]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMEN
10	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	H
9	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	Admi
13	Abhimanyu	Verma	800000	24-06-11 09.00.00	Admi
12	Abhimanyu K	Verma	800000	24-06-11 09.00.00	masterAdminfu
4	Amitabh	Singh	500000	21-02-20 09.00.00	Admi
8	Geetika	Chauhan	90000	21-04-11 09.00.00	Admi
1	Monika	Arora	100000	21-02-20 09.00.00	H
2	Niharika	Verma	80000	21-06-11 09.00.00	Admi
11	Raj	Arora	1000000	24-02-20 09.00.00	H
7	Satish	Kumar	75000	21-01-20 09.00.00	Accour
6	Vipul	Diwan	200000	21-06-11 09.00.00	Accour
3	Vishal	Singhal	300000	21-02-20 09.00.00	H
5	Vivek	Bhati	500000	21-06-11 09.00.00	Admi

Q-13. Write an SQL query to print details for Workers with the first names “Vipul” and “Satish” from the Worker table.

Hint: To get details for workers with specific first names, you can use the IN operator to check for multiple values.

In [24]:
```
%%sql
Select * from Worker where FIRST_NAME IN ('Vipul','Amitabh');
```

```
 * sqlite://
Done.
```

Out[24]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMENT
4	Amitabh	Singh	500000	21-02-20 09.00.00	Admin
6	Vipul	Diwan	200000	21-06-11 09.00.00	Account

Q-14. Write an SQL query to print details of workers excluding first names, “Vipul” and “Satish” from the Worker table.

Hint: To get details of workers except for those with specific first names, you can use the NOT IN operator to exclude certain values

In [25]:
```
%%sql
Select * from Worker where FIRST_NAME not in ('Vipul','Satish');
```

```
 * sqlite://
Done.
```

Out[25]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMEN
1	Monika	Arora	100000	21-02-20 09.00.00	H
2	Niharika	Verma	80000	21-06-11 09.00.00	Admi
3	Vishal	Singhal	300000	21-02-20 09.00.00	H
4	Amitabh	Singh	500000	21-02-20 09.00.00	Admi
5	Vivek	Bhati	500000	21-06-11 09.00.00	Admi
8	Geetika	Chauhan	90000	21-04-11 09.00.00	Admi
9	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	Admi
10	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	H
11	Raj	Arora	1000000	24-02-20 09.00.00	H
12	Abhimanyu K	Verma	800000	24-06-11 09.00.00	masterAdminfu
13	Abhimanyu	Verma	800000	24-06-11 09.00.00	Admi

Q-15. Write an SQL query to print details of Workers with DEPARTMENT name as “Admin”.

Hint: If you want to search for workers in a department that contains ‘Admin’ (in any part of the name), you can use LIKE with % to match any characters before or after ‘Admin

In [26]:
```
%%sql
Select * from Worker where DEPARTMENT like 'Admin%';
```

```
 * sqlite://
Done.
```

Out[26]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMENT
2	Niharika	Verma	80000	21-06-11 09.00.00	Admin
4	Amitabh	Singh	500000	21-02-20 09.00.00	Admin
5	Vivek	Bhati	500000	21-06-11 09.00.00	Admin
8	Geetika	Chauhan	90000	21-04-11 09.00.00	Admin
9	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	Admin
13	Abhimanyu	Verma	800000	24-06-11 09.00.00	Admin

Q-16. Write an SQL query to print details of the Workers whose FIRST_NAME contains ‘a’.

In [27]:
```
%%sql
Select * from Worker where FIRST_NAME like '%a%';
```

```
 * sqlite://
Done.
```

Out[27]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMEN
1	Monika	Arora	100000	21-02-20 09.00.00	H
2	Niharika	Verma	80000	21-06-11 09.00.00	Admi
3	Vishal	Singhal	300000	21-02-20 09.00.00	H
4	Amitabh	Singh	500000	21-02-20 09.00.00	Admi
7	Satish	Kumar	75000	21-01-20 09.00.00	Accour
8	Geetika	Chauhan	90000	21-04-11 09.00.00	Admi
9	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	Admi
10	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	H
11	Raj	Arora	1000000	24-02-20 09.00.00	H
12	Abhimanyu K	Verma	800000	24-06-11 09.00.00	masterAdminfu
13	Abhimanyu	Verma	800000	24-06-11 09.00.00	Admi

Q-17. Write an SQL query to print details of the Workers whose FIRST_NAME ends with ‘a’.

In [28]:
```
%%sql
Select * from Worker where FIRST_NAME like '%a';
```

```
 * sqlite://
Done.
```

Out[28]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMENT
1	Monika	Arora	100000	21-02-20 09.00.00	HR
2	Niharika	Verma	80000	21-06-11 09.00.00	Admin
8	Geetika	Chauhan	90000	21-04-11 09.00.00	Admin

Q-18. Write an SQL query to print details of the Workers whose FIRST_NAME ends with ‘h’ and contains six alphabets.

Hint: A total of 5 underscores followed by a letter h = 6 alphabets in total and ending with h

In [29]:
```
%%sql
Select * from Worker where FIRST_NAME like '_____h';
```

```
 * sqlite://
Done.
```

Out[29]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMENT
7	Satish	Kumar	75000	21-01-20 09.00.00	Account

Q-19. Write an SQL query to print details of the Workers whose SALARY lies between 100000 and 500000.

In [30]:
```
%%sql
Select * from Worker where SALARY between 100000 and 500000;
```

```
 * sqlite://
Done.
```

Out[30]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMENT
1	Monika	Arora	100000	21-02-20 09.00.00	HR
3	Vishal	Singhal	300000	21-02-20 09.00.00	HR
4	Amitabh	Singh	500000	21-02-20 09.00.00	Admin
5	Vivek	Bhati	500000	21-06-11 09.00.00	Admin
6	Vipul	Diwan	200000	21-06-11 09.00.00	Account

Q-20. Write an SQL query to print details of the Workers who joined in Feb 2021.

SELECT * FROM Worker WHERE YEAR(JOINING_DATE) = 2021 AND MONTH(JOINING_DATE) = 2;

In [31]:
```
%%sql
SELECT *
FROM Worker
WHERE strftime('%Y', JOINING_DATE) = '2011' AND strftime('%m', JOINING_DATE)
```

```
 * sqlite://
Done.
```

Out[31]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMENT

```
                        * Congratulations! You've completed
    module one.
```

At this point, you have acquired a good understanding of the basics of SQL, let’s move on to some more intermediate-level SQL query interview questions. These questions will require us to use more advanced SQL syntax and concepts, such as GROUP BY, HAVING, and ORDER BY. # Some intermediate SQL Interview Questions.

```
    Q-21. Write an SQL query to fetch the count of employees
    working in the department ‘Admin’.
```

In [32]:
```
%%sql
SELECT COUNT(*) FROM worker WHERE DEPARTMENT = 'Admin';
```

```
 * sqlite://
Done.
```

Out[32]:

COUNT(*)
6

```
    Q-22. SQL Query to Fetch Worker Names with Salaries >= 50000
    and <= 100000
```

In [33]:
```
%%sql
SELECT WORKER.FIRST_NAME || ' ' || LAST_NAME AS Worker_Name, Salary FROM Wor
WHERE Salary BETWEEN 50000 AND 100000;
```

```
 * sqlite://
Done.
```

Out[33]:

Worker_Name	SALARY
Monika Arora	100000
Niharika Verma	80000
Satish Kumar	75000
Geetika Chauhan	90000

```
    Q-23. Write SQL query to list Worker Count per department in
    descending order.
```

In [34]:
```
%%sql
SELECT DEPARTMENT, count(WORKER_ID) AS No_Of_Workers FROM Worker
GROUP BY DEPARTMENT
ORDER BY No_Of_Workers DESC;
```

```
 * sqlite://
Done.
```

Out[34]:

DEPARTMENT	No_Of_Workers
Admin	6
HR	4
Account	2
masterAdminful	1

Q-24. SQL Query to Print Worker Details Who Are Also Managers

In [35]:
```
%%sql
SELECT DISTINCT Worker.FIRST_NAME, Title.WORKER_TITLE
FROM Worker
INNER JOIN Title ON Worker.WORKER_ID = Title.WORKER_REF_ID
AND Title.WORKER_TITLE in ('Manager');
```

```
 * sqlite://
Done.
```

Out[35]:

FIRST_NAME	WORKER_TITLE
Monika	Manager
Vivek	Manager

In [36]:
```
%%sql
SELECT DISTINCT W.FIRST_NAME, T.WORKER_TITLE
FROM Worker W
INNER JOIN Title T ON W.WORKER_ID = T.WORKER_REF_ID AND T.WORKER_TITLE in ('
```

```
 * sqlite://
Done.
```

Out[36]:

FIRST_NAME	WORKER_TITLE
Monika	Manager
Vivek	Manager

Q-25. SQL Query to Fetch Duplicate Records with Matching Data in Specific Fields of a Table

In [37]:
```
%%sql
SELECT WORKER_TITLE, AFFECTED_FROM, COUNT(*) FROM Title
GROUP BY WORKER_TITLE,
AFFECTED_FROM HAVING COUNT(*) > 1;
```

```
 * sqlite://
Done.
```

Out[37]:

WORKER_TITLE	AFFECTED_FROM	COUNT(*)
Executive	2023-06-11 00:00:00	3
Lead	2023-06-11 00:00:00	2

Q-26. SQL Query to Show Only Odd Rows from a Table

In [38]:
```
%%sql
SELECT * FROM Worker
WHERE WORKER_ID % 2 <> 0;
```

```
 * sqlite://
Done.
```

Out[38]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMENT
1	Monika	Arora	100000	21-02-20 09.00.00	HR
3	Vishal	Singhal	300000	21-02-20 09.00.00	HR
5	Vivek	Bhati	500000	21-06-11 09.00.00	Admir
7	Satish	Kumar	75000	21-01-20 09.00.00	Account
9	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	Admir
11	Raj	Arora	1000000	24-02-20 09.00.00	HR
13	Abhimanyu	Verma	800000	24-06-11 09.00.00	Admir

Q-27. SQL Query to Show Only Even Rows from a Table

In [39]:
```
%%sql
SELECT * FROM Worker
WHERE WORKER_ID % 2 = 0;
```

```
 * sqlite://
Done.
```

Out[39]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMEN
2	Niharika	Verma	80000	21-06-11 09.00.00	Admi
4	Amitabh	Singh	500000	21-02-20 09.00.00	Admi
6	Vipul	Diwan	200000	21-06-11 09.00.00	Accour
8	Geetika	Chauhan	90000	21-04-11 09.00.00	Admi
10	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	H
12	Abhimanyu K	Verma	800000	24-06-11 09.00.00	masterAdminfu

Q-28. SQL Query to Clone a New Table from Another Table

In [40]:
```
%%sql
CREATE TABLE WorkerClone AS SELECT * FROM Worker;
```

```
 * sqlite://
Done.
```

Out[40]: []

Q-29. SQL Query to Display Intersecting Records of Two Tables

In [41]:
```
%%sql
SELECT * FROM Worker
INTERSECT
SELECT * FROM WorkerClone;
```

```
 * sqlite://
Done.
```

Out[41]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMEN
1	Monika	Arora	100000	21-02-20 09.00.00	H
2	Niharika	Verma	80000	21-06-11 09.00.00	Admi
3	Vishal	Singhal	300000	21-02-20 09.00.00	H
4	Amitabh	Singh	500000	21-02-20 09.00.00	Admi
5	Vivek	Bhati	500000	21-06-11 09.00.00	Admi
6	Vipul	Diwan	200000	21-06-11 09.00.00	Accour
7	Satish	Kumar	75000	21-01-20 09.00.00	Accour
8	Geetika	Chauhan	90000	21-04-11 09.00.00	Admi
9	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	Admi
10	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	H
11	Raj	Arora	1000000	24-02-20 09.00.00	H
12	Abhimanyu K	Verma	800000	24-06-11 09.00.00	masterAdminfu
13	Abhimanyu	Verma	800000	24-06-11 09.00.00	Admi

Q-30. SQL Query to Show Records from One Table That Are Not Present in Another Table

In [42]:
```
%%sql
SELECT * FROM Worker EXCEPT SELECT * FROM WorkerClone;
```

```
 * sqlite://
Done.
```

Out[42]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMENT

Q-31. SQL Query to Show the Current Date and Time

In [43]:
```
%%sql
SELECT CURRENT_TIMESTAMP;
```

```
 * sqlite://
Done.
```

Out[43]:

CURRENT_TIMESTAMP
2025-02-06 07:34:40

Q-32. SQL Query to Show the Top n (say 10) Records of a Table

In [44]:
```
%%sql
SELECT *
FROM Worker
ORDER BY SALARY DESC  -- Order by salary in descending order to get the high
LIMIT 10;             -- Limit the result to the top 10 records
```

```
 * sqlite://
Done.
```

Out[44]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMEN
9	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	Admi
10	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	H
11	Raj	Arora	1000000	24-02-20 09.00.00	H
12	Abhimanyu K	Verma	800000	24-06-11 09.00.00	masterAdminfu
13	Abhimanyu	Verma	800000	24-06-11 09.00.00	Admi
4	Amitabh	Singh	500000	21-02-20 09.00.00	Admi
5	Vivek	Bhati	500000	21-06-11 09.00.00	Admi
3	Vishal	Singhal	300000	21-02-20 09.00.00	H
6	Vipul	Diwan	200000	21-06-11 09.00.00	Accour
1	Monika	Arora	100000	21-02-20 09.00.00	H

* Congratulations! You've completed module Two.*

* NOW, Some ADVANCE SQL Interview Questions. *

These interview questions involve queries with more complex SQL syntax and concepts, such as nested queries, joins, unions, and intersects.

Q-33. SQL Query to Determine the Nth (say n=5) Highest Salary

Hint: To find the nth highest salary (like the 5th highest), you can use a combination of ORDER BY and LIMIT, or use a subquery with ROW_NUMBER to rank the salaries

```
        * Solution 1: General Approach | Universal Approach

SELECT TOP 1 Salary
FROM (
 SELECT DISTINCT TOP 5 Salary
 FROM Worker
 ORDER BY Salary DESC
) AS Result
ORDER BY Salary ASC;
```

Output:The 5th highest salary should be 300,000 (from Vishal Singhal).

```
Explanation:
Step 1 Inner Query: (SELECT DISTINCT TOP 5 Salary FROM Worker
ORDER BY Salary DESC) will select the top 5 highest distinct
salaries in decreasing order.

Step 2. Wrapping this query in outer query "SELECT TOP 1
Salary ORDER BY Salary ASC" will then orders these salaries
in ascending order and return the top salary, which is
effectively giving you the 5th highest salary


        * Solution 2: For MySQL, sqlite3 or PostgreSQL (using
LIMIT with OFFSET):
```

Instead, we can use a subquery with LIMIT and OFFSET to achieve the same result.

Here’s an example to get the third highest salary:

```
%%sql
SELECT DISTINCT SALARY
FROM Worker
ORDER BY SALARY DESC
LIMIT 1 OFFSET 2;

Explanation:
In this query:
~ ORDER BY Salary DESC sorts the salaries in descending
order.
~ LIMIT 1 specifies that we want to return one row.
~ OFFSET 2 skips the first two rows (the highest and second
highest salaries) and returns the third highest salary
```

* Solution 3: For SQL Server (using ROW_NUMBER()):

```
WITH RankedSalaries AS (
SELECT Salary,
       ROW_NUMBER() OVER (ORDER BY Salary DESC) AS Rank
FROM Worker
)
SELECT Salary
FROM RankedSalaries
WHERE Rank = 5;
```

Explanation: This query uses the ROW_NUMBER() function to assign a unique row number to each salary based on descending order.

Then, the result is filtered for the salary where Rank = n

In [45]:
```
%%sql
SELECT Salary
FROM (
  SELECT DISTINCT Salary
  FROM Worker
  ORDER BY Salary DESC
) AS DistinctSalaries
ORDER BY Salary DESC
LIMIT 1 OFFSET 4;
```

```
 * sqlite://
Done.
```

Out[45]:

Salary
300000

In [46]:
```
%%sql
SELECT DISTINCT SALARY
FROM Worker
ORDER BY SALARY DESC
LIMIT 1 OFFSET 4;
```

```
 * sqlite://
Done.
```

Out[46]:

SALARY
300000

Q-34. SQL Query to Determine 5th Highest Salary Without Using TOP or Limit

Solution:

```
SELECT Salary
```

```
FROM Worker W1
WHERE (SELECT COUNT(DISTINCT W2.Salary)
        FROM Worker W2
        WHERE W2.Salary > W1.Salary) = n - 1;


In this subquery
SELECT COUNT(DISTINCT W2.Salary) FROM Worker W2
WHERE W2.Salary > W1.Salary)
counts the number of distinct salaries that are greater than
the current salary in W1.
```

By setting this equal to n - 1, we find the salary in W1 that has exactly n - 1 distinct salaries greater than it, which effectively gives us the nth highest salary.

Just replace n with the specific value you need to find the nth highest salary.

In [47]:
```
%%sql
SELECT Salary FROM Worker W1
WHERE (SELECT COUNT(DISTINCT W2.Salary) FROM Worker W2 WHERE W2.Salary >= W1
```

```
 * sqlite://
Done.
```

Out[47]:

SALARY
500000
500000

```
Q-35. SQL Query to Fetch the List of Employees with the Same
Salary
```

Solution 1:

```
SELECT WORKER_ID, FIRST_NAME, LAST_NAME, SALARY
FROM Worker
WHERE SALARY IN (
    SELECT SALARY
    FROM Worker
    GROUP BY SALARY
    HAVING COUNT(SALARY) > 1
    )
ORDER BY SALARY DESC;

Explanation:
Subquery (SELECT SALARY FROM Worker GROUP BY SALARY HAVING
COUNT(SALARY) > 1):

This part groups the workers by their salary and selects only
those salaries that appear more than once (i.e., those with a
COUNT(SALARY) > 1).
```

Main Query (SELECT WORKER_ID, FIRST_NAME, LAST_NAME, SALARY FROM Worker WHERE SALARY IN (...)):

The main query then fetches the list of workers whose salary is in the list of salaries returned by the subquery. It ensures that only employees with the same salary are included in the result.

In [59]:
```
%%sql
SELECT WORKER_ID, FIRST_NAME, LAST_NAME, SALARY
FROM Worker
WHERE SALARY IN (
    SELECT SALARY
    FROM Worker
    GROUP BY SALARY
    HAVING COUNT(SALARY) > 1
)
ORDER BY SALARY DESC;
```

```
 * sqlite://
Done.
```

Out[59]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY
9	ABHISHEK	RAJ	10000000
10	ABHISHEK	RAJ	10000000
12	Abhimanyu K	Verma	800000
13	Abhimanyu	Verma	800000
4	Amitabh	Singh	500000
5	Vivek	Bhati	500000

SOLUTION 2: (Use of implicit joins using comma) To find employees with the same salary,

1. Use a self-join to compare the SALARY from two instances of the Worker table.
2. Then, make sure the WORKER_IDs are different to avoid comparing the same person to themselves.
3. Finally, use DISTINCT to eliminate duplicates and get unique results.

```
%%sql
SELECT distinct W.WORKER_ID, W.FIRST_NAME, W.Salary
from Worker W,
Worker W1
where W.Salary = W1.Salary and W.WORKER_ID != W1.WORKER_ID;
```

Note: The use of implicit joins (using a comma , to separate tables) is considered an outdated style. It's better to use explicit JOIN syntax for clarity

and performance.

In [60]:
```
%%sql
        SELECT distinct W.WORKER_ID, W.FIRST_NAME, W.Salary
        from Worker W,
        Worker W1
        where W.Salary = W1.Salary and W.WORKER_ID != W1.WORKER_ID;
```

```
 * sqlite://
Done.
```

Out[60]:

WORKER_ID	FIRST_NAME	SALARY
4	Amitabh	500000
5	Vivek	500000
9	ABHISHEK	10000000
10	ABHISHEK	10000000
12	Abhimanyu K	800000
13	Abhimanyu	800000

SOLUTION 3 (Use of explicit JOIN by mentioning syntax)

```
%%sql
SELECT DISTINCT W.WORKER_ID, W.FIRST_NAME, W.Salary
FROM Worker W
JOIN Worker W1 ON W.Salary = W1.Salary
WHERE W.WORKER_ID != W1.WORKER_ID;
```

```
WORKER W
JOIN Worker W1 ON W.Salary = W1.Salary:
```

Explanation: This performs an explicit join between the Worker table (aliased as W) and itself (aliased as W1), matching rows where the salary is the same.

```
WHERE W.WORKER_ID != W1.WORKER_ID:
```

Explanation: This condition ensures that the query does not match the same worker to themselves.

```
SELECT DISTINCT W.WORKER_ID, W.FIRST_NAME, W.Salary:
```

Explanation: The DISTINCT keyword ensures that the query only returns unique combinations of worker IDs, first names, and salaries, preventing duplicate rows from appearing.

In [61]:
```
%%sql
    SELECT DISTINCT W.WORKER_ID, W.FIRST_NAME, W.Salary
    FROM Worker W
    JOIN Worker W1 ON W.Salary = W1.Salary
    WHERE W.WORKER_ID != W1.WORKER_ID;
```

```
 * sqlite://
Done.
```

Out[61]:

WORKER_ID	FIRST_NAME	SALARY
4	Amitabh	500000
5	Vivek	500000
9	ABHISHEK	10000000
10	ABHISHEK	10000000
12	Abhimanyu K	800000
13	Abhimanyu	800000

36. SQL Query to List the Employee with the Second-Highest Salary

To find the second-highest salary, use a subquery to get the highest salary, and then find the maximum salary that is less than that. This way, you'll get the second-highest value.

In [63]:
```
%%sql
SELECT max(Salary) from Worker
where Salary not in (Select max(Salary) from Worker);
```

```
 * sqlite://
Done.
```

Out[63]:

max(Salary)
1000000

37. SQL Query to Display One Row Twice in the Results from a Table

Hint: To show one row twice, use the UNION ALL operator to select the same data from the table twice. Unlike UNION, UNION ALL doesn't remove duplicates, so the row will appear two times in the result.

EXPLANATION:Your SQL query performs the following:

1. It selects the FIRST_NAME and DEPARTMENT columns from the Worker table where the DEPARTMENT is 'HR'.

2. It then uses UNION ALL to combine that result with another select statement that also selects the FIRST_NAME and DEPARTMENT columns from the Worker table where the DEPARTMENT is 'HR'.
3. However, the use of UNION ALL means that the result will include duplicate records (if any) that match the criteria in both queries.

Since both queries are identical in the columns selected and the condition (DEPARTMENT = 'HR'), the result will be the list of workers in the HR department, but the list will be duplicated.

In [62]:
```
%%sql
SELECT FIRST_NAME, DEPARTMENT from Worker W where W.DEPARTMENT='HR'
union all
select FIRST_NAME, DEPARTMENT from Worker W1 where W1.DEPARTMENT='HR';
```

```
 * sqlite://
Done.
```

Out[62]:

FIRST_NAME	DEPARTMENT
Monika	HR
Vishal	HR
ABHISHEK	HR
Raj	HR
Monika	HR
Vishal	HR
ABHISHEK	HR
Raj	HR

38. SQL Query to Fetch Intersecting Records of Two Tables

-- Note: -- You Must Run Query #28 -- Both Tables Must Have Same Structure.

```
        -- SQLite and MySQL compatible syntax
SELECT * FROM Worker
INTERSECT
SELECT * FROM WorkerClone;

         -- SQLite and MySQL compatible syntax
SELECT w.*
FROM Worker w
INNER JOIN WorkerClone wc ON w.WORKER_ID = wc.WORKER_ID;
```

In [64]:
```
%%sql
SELECT * FROM Worker
```

```
INTERSECT
SELECT * FROM WorkerClone;
```

```
 * sqlite://
Done.
```

Out[64]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMEN
1	Monika	Arora	100000	21-02-20 09.00.00	H
2	Niharika	Verma	80000	21-06-11 09.00.00	Admi
3	Vishal	Singhal	300000	21-02-20 09.00.00	H
4	Amitabh	Singh	500000	21-02-20 09.00.00	Admi
5	Vivek	Bhati	500000	21-06-11 09.00.00	Admi
6	Vipul	Diwan	200000	21-06-11 09.00.00	Accour
7	Satish	Kumar	75000	21-01-20 09.00.00	Accour
8	Geetika	Chauhan	90000	21-04-11 09.00.00	Admi
9	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	Admi
10	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	H
11	Raj	Arora	1000000	24-02-20 09.00.00	H
12	Abhimanyu K	Verma	800000	24-06-11 09.00.00	masterAdminfu
13	Abhimanyu	Verma	800000	24-06-11 09.00.00	Admi

In [65]:

```
%%sql
SELECT w.*
FROM Worker w
INNER JOIN WorkerClone wc ON w.WORKER_ID = wc.WORKER_ID;
```

```
 * sqlite://
Done.
```

Out[65]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMEN
1	Monika	Arora	100000	21-02-20 09.00.00	H
2	Niharika	Verma	80000	21-06-11 09.00.00	Admi
3	Vishal	Singhal	300000	21-02-20 09.00.00	H
4	Amitabh	Singh	500000	21-02-20 09.00.00	Admi
5	Vivek	Bhati	500000	21-06-11 09.00.00	Admi
6	Vipul	Diwan	200000	21-06-11 09.00.00	Accour
7	Satish	Kumar	75000	21-01-20 09.00.00	Accour
8	Geetika	Chauhan	90000	21-04-11 09.00.00	Admi
9	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	Admi
10	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	H
11	Raj	Arora	1000000	24-02-20 09.00.00	H
12	Abhimanyu K	Verma	800000	24-06-11 09.00.00	masterAdminfu
13	Abhimanyu	Verma	800000	24-06-11 09.00.00	Admi

39. SQL Query to Fetch the First 50% of Records from a Table

To fetch the first 50% of records from a table, you can use a LIMIT clause in combination with a subquery that counts the total number of rows. By dividing this count by 2, you can retrieve the first half of the records.

In [66]:
```
%%sql
SELECT *
FROM WORKER
WHERE WORKER_ID <= (SELECT count(WORKER_ID)/2 from Worker);
```

```
 * sqlite://
Done.
```

Out[66]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMENT
1	Monika	Arora	100000	21-02-20 09.00.00	HR
2	Niharika	Verma	80000	21-06-11 09.00.00	Admin
3	Vishal	Singhal	300000	21-02-20 09.00.00	HR
4	Amitabh	Singh	500000	21-02-20 09.00.00	Admin
5	Vivek	Bhati	500000	21-06-11 09.00.00	Admin
6	Vipul	Diwan	200000	21-06-11 09.00.00	Account

40. SQL Query to Fetch Departments with Less than Five People in Them

To find departments with fewer than five people, use a GROUP BY clause to group by the department, then use HAVING to filter departments where the count of employees is less than 5.

In [67]:
```
%%sql
SELECT DEPARTMENT, COUNT(WORKER_ID) as 'Number of Workers'
FROM Worker
GROUP BY DEPARTMENT HAVING COUNT(WORKER_ID) < 5;
```

```
 * sqlite://
Done.
```

Out[67]:

DEPARTMENT	Number of Workers
Account	2
HR	4
masterAdminful	1

41. SQL Query to Show All Departments with the Number of People in There

To show all departments with the number of people in each, use a GROUP BY clause to group by the department, and then use the COUNT function to count the number of employees in each department.

In [68]:
```
%%sql
SELECT DEPARTMENT, COUNT(DEPARTMENT) as 'Number of Workers'
FROM Worker
GROUP BY DEPARTMENT;
```

```
 * sqlite://
Done.
```

Out[68]:

DEPARTMENT	Number of Workers
Account	2
Admin	6
HR	4
masterAdminful	1

42. SQL Query to Show the Last Record from a Table

To fetch the last record using the MAX() function, use a subquery to get the max WORKER_ID and then filter the main query to return the row matching that WORKER_ID.

In [69]:
```
%%sql
Select * from Worker
where WORKER_ID = (SELECT max(WORKER_ID) from Worker);
```

```
 * sqlite://
Done.
```

Out[69]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMENT
13	Abhimanyu	Verma	800000	24-06-11 09.00.00	Admin

43. SQL Query to Fetch the First Row of a Table

To fetch the first row from a table using MIN(), you can use a subquery to get the min value of a column (such as WORKER_ID) and then filter the main query to return the row matching that value.

In [70]:
```
%%sql
Select * from Worker
where WORKER_ID = (SELECT min(WORKER_ID) from Worker);
```

```
 * sqlite://
Done.
```

Out[70]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMENT
1	Monika	Arora	100000	21-02-20 09.00.00	HR

44. SQL Query to Fetch the Last Five Records from a Table

To fetch the last five records, you can order the table by a column (like WORKER_ID or a timestamp) in descending order and limit the result to five rows.

```
-- Solution 1:

SELECT * FROM Worker
ORDER BY WORKER_ID DESC LIMIT 5;

-- Solution 2: If you are using a database that does not
support the LIMIT keyword (e.g., SQL Server), you could use
TOP in a slightly different way

SELECT TOP 5 *
FROM Worker
ORDER BY WORKER_ID DESC;
```

In [71]:
```
%%sql
SELECT * FROM Worker
ORDER BY WORKER_ID DESC LIMIT 5;
```

```
 * sqlite://
Done.
```

Out[71]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMEN
13	Abhimanyu	Verma	800000	24-06-11 09.00.00	Admi
12	Abhimanyu K	Verma	800000	24-06-11 09.00.00	masterAdminfu
11	Raj	Arora	1000000	24-02-20 09.00.00	H
10	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	H
9	ABHISHEK	RAJ	10000000	21-04-11 09.00.00	Admi

The most straightforward way to get the first 5 records is to either:

1. Use LIMIT in MySQL/PostgreSQL.

```
SELECT * FROM Worker
ORDER BY WORKER_ID ASC LIMIT 5;
```

2. Use TOP in SQL Server. If you're using SQL Server, you can use TOP instead:

```
SELECT TOP 5 *
FROM Worker
ORDER BY WORKER_ID ASC;
```

3. Example: Using ROW_NUMBER() (works in many SQL databases like PostgreSQL, SQL Server, MySQL):

```
SELECT *
 FROM (
     SELECT *, ROW_NUMBER() OVER (ORDER BY WORKER_ID ASC)
AS row_num
     FROM Worker
 ) AS W
 WHERE row_num <= 5;
```

Explanation:

1. ROW_NUMBER(): This assigns a unique row number to each record ordered by WORKER_ID in ascending order.
2. WHERE row_num <= 5: Filters out only the first 5 records (based on ascending WORKER_ID).

In [72]:
```
%%sql
SELECT *
FROM Worker
WHERE WORKER_ID <= 5
ORDER BY WORKER_ID ASC
LIMIT 5;
```

```
 * sqlite://
Done.
```

Out[72]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMENT
1	Monika	Arora	100000	21-02-20 09.00.00	HR
2	Niharika	Verma	80000	21-06-11 09.00.00	Admin
3	Vishal	Singhal	300000	21-02-20 09.00.00	HR
4	Amitabh	Singh	500000	21-02-20 09.00.00	Admin
5	Vivek	Bhati	500000	21-06-11 09.00.00	Admin

In [73]:
```
%%sql
SELECT *
FROM (
    SELECT *, ROW_NUMBER() OVER (ORDER BY WORKER_ID ASC) AS row_num
    FROM Worker
) AS W
WHERE row_num <= 5;
```

```
 * sqlite://
Done.
```

Out[73]:

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMENT
1	Monika	Arora	100000	21-02-20 09.00.00	HR
2	Niharika	Verma	80000	21-06-11 09.00.00	Admin
3	Vishal	Singhal	300000	21-02-20 09.00.00	HR
4	Amitabh	Singh	500000	21-02-20 09.00.00	Admin
5	Vivek	Bhati	500000	21-06-11 09.00.00	Admin

45. SQL Query to Show Employees with the Highest Salary in Each Department

To get the employees with the highest salary in each department, you can use a subquery to find the maximum salary per department and then join it with the main table to fetch the employee names and salaries.

In [74]:
```
%%sql
SELECT t.DEPARTMENT, t.FIRST_NAME, t.Salary
from (SELECT max(Salary) as TotalSalary, DEPARTMENT from Worker group by DEP
Inner Join Worker t on TempNew.DEPARTMENT = t.DEPARTMENT and TempNew.TotalSa
```

```
 * sqlite://
Done.
```

Out[74]:

DEPARTMENT	FIRST_NAME	SALARY
Account	Vipul	200000
Admin	ABHISHEK	10000000
HR	ABHISHEK	10000000
masterAdminful	Abhimanyu K	800000

46. SQL Query to Fetch the Top Three Max Salaries from a Table

To fetch the top three max salaries, you can use a subquery that counts how many distinct salaries are greater than or equal to each salary, and then filter it to get the top three.

In [76]:
```
%%sql
SELECT distinct Salary
from Worker a WHERE 3 >= (SELECT count(distinct Salary)
                          from Worker b WHERE a.Salary <= b.Salary)
order by a.Salary desc;
```

```
 * sqlite://
Done.
```

Out[76]:

SALARY
10000000
1000000
800000

47. SQL Query to Fetch the Three Min Salaries from a Table

To fetch the three min salaries, you can do the same as we did for the max salaries. But, this time order the salaries in ascending order and limit the result to three rows.

In [77]:
```
%%sql
SELECT distinct Salary
from Worker a WHERE 3 >= (SELECT count(distinct Salary)
                          from Worker b WHERE a.Salary >= b.Salary)
order by a.Salary desc;
```

```
 * sqlite://
Done.
```

Out[77]:

SALARY
90000
80000
75000

48. SQL Query to Fetch the Nth Max Salaries from a Table

To fetch the nth max salary using a variable, you can define the value of n using a WITH clause and then use it in the query to limit the results.

```
                -- Set the value of n
WITH vars AS (SELECT 5 AS n)

                -- Use the variable in a query
SELECT DISTINCT Salary FROM Worker a
WHERE (SELECT n FROM vars) >= (SELECT COUNT(DISTINCT Salary)
FROM Worker b
                                WHERE a.Salary <= b.Salary)
ORDER BY a.Salary DESC;
```

In [78]:
```
%%sql
WITH vars AS (SELECT 5 AS n)

SELECT DISTINCT Salary FROM Worker a
```

```
WHERE (SELECT n FROM vars) >= (SELECT COUNT(DISTINCT Salary) FROM Worker b
                               WHERE a.Salary <= b.Salary)
ORDER BY a.Salary DESC;
```

```
 * sqlite://
Done.
```

Out[78]:

SALARY
10000000
1000000
800000
500000
300000

49. SQL Query to Fetch Departments and Their Total Salaries

To fetch departments along with the total salaries, you can group the data by department and sum the salaries for each group.

In [79]:
```
%%sql
SELECT DEPARTMENT, sum(Salary) from Worker
group by DEPARTMENT;
```

```
 * sqlite://
Done.
```

Out[79]:

DEPARTMENT	sum(Salary)
Account	275000
Admin	11970000
HR	11400000
masterAdminful	800000

50. SQL Query to Fetch Workers with the Highest Salary

To fetch departments along with the total salaries, you can group the data by department and sum the salaries for each group.

In [80]:
```
%%sql
SELECT FIRST_NAME, SALARY from Worker
WHERE SALARY=(SELECT max(SALARY) from Worker);
```

```
 * sqlite://
Done.
```

Out[80]:

FIRST_NAME	SALARY
ABHISHEK	10000000
ABHISHEK	10000000